

Figure 4-6: In this DAG representation of the maze, nodes represent intersections, and edges represent the north, south, east, or west path from the intersection. Some nodes have lowercase letters to correspond to intersections in [Figure 4-5](#).

Because of this similarity in structure, we can use a tree traversal algorithm to solve the maze. The nodes in this tree graph represent intersections where the maze solver could choose one of the north, south, east, or west paths to follow to the next intersection. The root node is the start of the maze, and the leaf nodes represent dead ends.

The recursive case occurs when the tree traversal algorithm moves from one node to the next. If the tree traversal reaches a leaf node (a dead end in the maze), the algorithm has reached a base case and must backtrack to an earlier node and follow a different path. Once the algorithm reaches the exit node, the path it took from the root node represents the maze solution. Let's ask our three recursive algorithm questions about the maze-solving algorithm:

What is the base case? Reaching a dead end or the exit of the maze.

What argument is passed to the recursive function call? The x, y coordinates, along with the maze data and list of already visited x, y coordinates.

How does this argument become closer to the base case? Like the flood fill algorithm, the x, y coordinates keep moving to neighboring coordinates until they eventually reach dead ends or the final exit.

This *mazeSolver.py* program contains the Python code for solving the maze stored in the `MAZE` variable:

Python

```
# Create the maze data structure:
# You can copy-paste this from inventwithpython.com/examplemaze.txt
MAZE = """
#####
#S#          #      # #  #      #      #      #
# ##### ##### # #### # # # # # #### # # ##### # #### # # #
# #      #      #      # # # # # # # #      # # # #      # # #
# # # ##### # ##### ##### # ##### ##### ##### # # ##### #### # #
#  #      # # # #      # #  # #      #      #  #  #  #  #  # #
##### # # # ##### # #### # ##### ##### ##### # # ##### ##### ####

```